

functional Web app – frame generators, access lists, templating, etc – so your team can assemble the page or app as per requirements. This is an alluring option for teams that are attempting to build a product quickly, since it enables them to focus only on the development of the app and not everything else that revolves around it. In any case, if you have sophisticated custom requirements or are already working with different types of custom software, you can't exploit those libraries.

Full-stack frameworks

There are a whole new set of full-stack options available among Python frameworks. Some of the prominent frameworks are TurboGears 2, Pylons, and Web2py. But the most popular one among them is Django.

Django: This is the most well-known Python framework and anyone can easily justify the reason behind it. A huge number of websites already use Django, from publishing houses to social media and sharing sites to significant establishments and non-profits. Since Django was initially developed for use in the newsroom, it's not a big deal that dailies like the *Washington Post* and *The Guardian* work on this framework. New companies and startups like Eventbrite and Disqus have migrated to Django to improve conversion rates, while social media giants like Instagram and Pinterest have used it to control their dynamic Web apps.

When considered as a framework, Django is known for being quick to build and friendly for amateur developers. It's a 'batteries included' framework, which means it supplies all the basic components you require such as authentication, rendering template, ORM, routing, etc. It's also well documented, which isn't the case with some other mainstream Web frameworks.

Django can considerably reduce the time taken to bootstrap a new project by dealing with a lot of choices. However, what you gain in speed can be lost in long-term flexibility. For instance, the inbuilt ORM of Django works efficiently in major scenarios. However, it's not as ground breaking as SQLAlchemy, which is also termed the best Python database abstraction tool. While you can hypothetically use SQLAlchemy with Django, you'll lose out on some major functionalities that make Django so appealing to start with.

Web2py: This is another prominent full-stack framework. One thing to remember about Web2py is that it isn't compatible with Python 3. The original developers behind Web2py have guaranteed a compatible successor for Python 3, but haven't launched it till now.

Even though it is ten years behind the most recent version of Python, Web2py is still used by many major enterprises, including multinational banks. What makes this older Web framework still engaging for developers is its unique functionalities. For one, it's as simple and easy to learn as

Django, and is more flexible and portable too. The same code can be employed for almost any VPS with a SQL database or MongoDB, regardless of whether AWS or Google App Engine are used.

Web2py has got intense support -- it is well-documented and has a strong community behind it. Another perfect element is that Web2py is accompanied by its very own IDE that incorporates a code editor, debugger, bug ticketing framework, a single tick deployment, etc. If your organisation is focused on Python 2 for the coming years or you intend to make use of some more established Python libraries and software, then Web2py could perfectly suit your requirements.

Pyramid: Pyramid isn't exactly a full-stack framework but bills itself as the Goldilocks framework. Pyramid is enriched with features without forcing any particular way of getting things done on users. It is lightweight without leaving you all alone as your app develops. It's one of the most loved Web frameworks among experienced Python developers on account of its transparency and modularity, and has been used by both moderately sized teams as well as tech giants like Mozilla, Yelp, SurveyMonkey and Dropbox.

In reality, almost every component of a Pyramid framework can be swapped out. You can pick how you interface with a database, or even what type (or types) of databases you want to connect with. It doesn't authorise certain choices for you like Django does, and it also discourages some 'magic' features which handle certain assignments automatically. It also doesn't behave in an anticipated or attractive way.

Pyramid is popular because of its security solutions, which make it simple to set up and to check access control records. Another innovative functionality worth mentioning is Pyramid's Traversal system for mapping URLs to code, which eventually makes it easy to build RESTful APIs.

Micro-frameworks

Consider the possibility that you don't need handholding or the sophistication of a full-stack framework. Nowadays, modern Web apps require lots of moving parts, including database abstraction, frame approval and modified access control records. But there are a lot of Web applications that don't require any of it. For such projects, a micro-framework might be exactly what's required.

These ultra-lightweight frameworks are developed to get dead Web apps up and running as fast as possible. Their capabilities are minimal by design– any functionality you could get by installing another library is intentionally left out. An advantage of working with this moderate approach is that your code can be cleaner and the site quicker. This is because micro-frameworks are less abstracted than full-stack structures. The code you compose will be significantly closer to HTTP capacities than with a more beginner-friendly framework.